

Escuela de Ciencias de la Computación
Final 2026-I
CC4P1 Programación Concurrente y Distribuida

Implementación de un Sistema Distribuido para el reconocimiento de objetos, animales o personas que requiera entrenamiento previo, con algoritmo de consenso.

El objetivo de esta evaluación es Diseñar e implementar un sistema distribuido que permita el entrenamiento y consumo de modelos de inteligencia artificial (IA) de manera paralela, distribuida y concurrente, garantizando persistencia, accesibilidad y tolerancia a fallos mediante el algoritmo de consenso Raft.

Desarrollar previamente el Modelos con sus pesos de IA Distribuido (Servidor de Entrenamiento)

Se debe desarrollar los modelos y los pesos de un sistema distribuido en el cual este preparado para los clientes envíe datos para el testeo de entrada (inputs) a un conjunto de servidores (nodos). Estos servidores pueden ser manera secuencial (se considerará si se procesa en paralelo distribuido tipo cpu), gestionando los recursos de manera distribuida y concurrente.

Requisitos:

- La parte del entrenamiento es de acuerdo al modelo del grupo.
- La carga de trabajo puede ser secuencial, en caso que sea paralelo distribuido la carga de trabajo debe distribuirse entre los nodos del sistema para optimizar el proceso de entrenamiento.
- Se debe garantizar la persistencia y accesibilidad de los modelos entrenados para su posterior consumo.
- Se tiene que entrenar con un grupo "n" de objetos, animales o personas, que se pueda reconocer, se tomará en cuenta si mayor es el n para la evaluación, por ejemplo, n=2 puede reconocer perros y gatos.



Consumo de Modelos de IA (Servidor de Testeo [web o desktop remoto])

El sistema debe permitir que la cámara enésima use un modelo de IA previamente entrenado y según **identifique los objetos y guarde una imagen** (objetos, animales o personas) **en un archivo y agregue en un registro que objeto se acercó y su imagen.**

Requisitos:

- El servidor de testeo de objeto debe poder usar su modelo de IA y autónomamente reconocer los objetos entrenados.

- El modelo debe procesar la entrada de los frames del video y devolver la salida esperada de manera eficiente por cada cámara.
- Escalabilidad, para un numero “c” de cámaras, mientras más cámaras se considerará
- Uso del algoritmo Raft, para garantizar si un nodo falla, los demás continúen funcionando y mantengan consistencia en el registro.



Detecto un:

Carro



Detecto:

Naranja



Detecto:

Maria

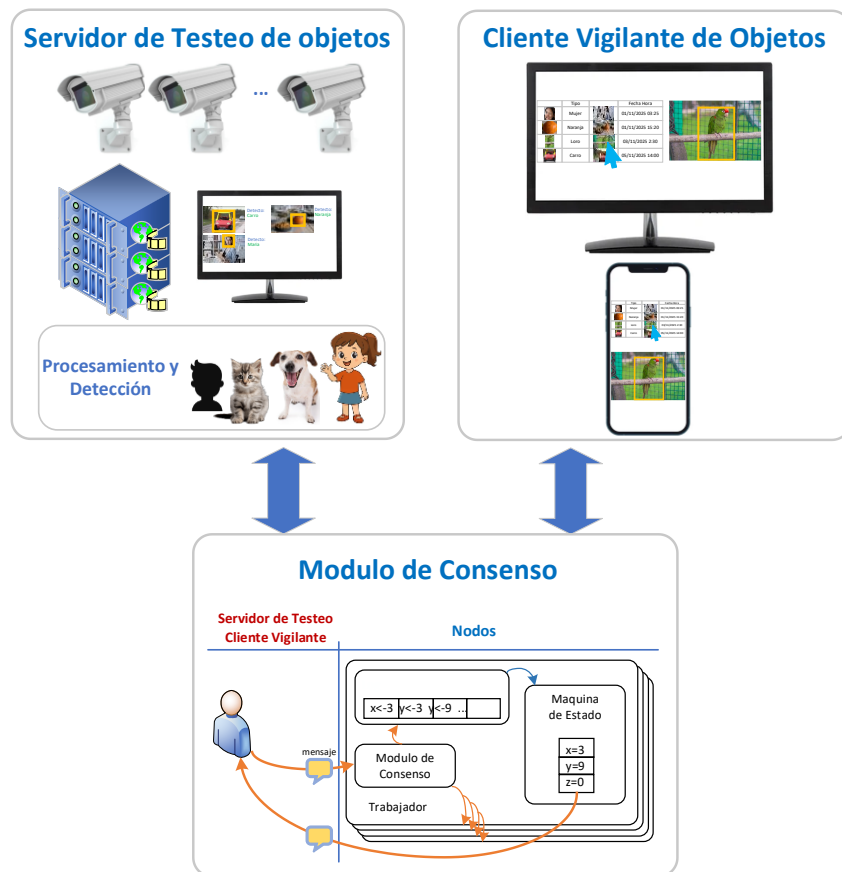
Vigilante de objetos (Cliente Vigilante en [web o desktop remoto] y móvil)

El cliente vigilante de objetos puede ver visualmente el **registro que objetos identificados, su foto, la fecha, hora de la captura y la cámara** enésima que lo proceso el **servidor de testeo** de manera continua.

Uso del algoritmo Raft, que asegura que el cliente mantenga un registro consistente y tolerante a fallos, incluso si algún servidor de testeo deja de responder.

Encontro	Tipo	Encontro	Fecha Hora
	Mujer		01/11/2025 03:25
	Naranja		01/11/2025 15:20
	Loro		03/11/2025 2:30
	Carro		05/11/2025 14:00





Consumo del cliente y de los modelos de IA con Algoritmo de Consenso Raft

El consumo del modelo debe realizarse con el Algoritmo de consenso Raft. En caso que un nodo falle los demás deben de seguir funcionando.

- Usar como mínimo 3 cámaras ip.
- Se solicita como mínimo un lenguaje de programación, y luego se puede adicionar más lenguajes de programación en $= \{LP1, LP2\}$, exponer y redactar un informe, por cada alumno adicional se agregará uno más $\{LP3, LP4, \dots\}$, los lenguajes de programación adicionales se tomarán en cuenta para la evaluación.
- Se iniciará los nodos servidor de video, servidor de testeo de objetos y servidor de entrenamiento de objetos, luego se inician los clientes y recibirá las peticiones de Clientes.
- El módulo del entrenamiento de IA puede estar en el lenguaje de programación que decida.
- Se iniciará los nodos "workers n" luego se inician los clientes y recibirá las peticiones de Clientes.
- Sólo el módulo del entrenamiento de IA como mínimo puede estar solo en Java versión mínima 8 u otro, los lenguajes de programación adicionales se tomarán en cuenta para la evaluación.
- Desplegar de manera local sin internet.
- Tomar como base las explicaciones y el código de clase.
- Usar como base las librerías de desarrollo base que viene con el lenguaje de programación base.

- Exponer y Ejecutar en cluster, con $= \{LP1, LP2, \dots\}$, para poder verificar el consenso.
- Sistema Operativos, SO1 y SO2, donde $SO1 \neq SO2$ y $SO1 = SO2 = \{LP1, LP2, \dots\}$.
- Graficar la arquitectura diseñada.
- Graficar el diagrama de protocolo.
- Usar solo Sockets de cada Lenguaje de programación.
- No usar websocket, socketio, frameworks, RabbitMQ, MQ, Librerías de Comunicación, etc.
- Explicar el desarrollo del programa puntualmente.
- Desplegar el programa en redes LAN y WIFI.
- Evaluar enfoque cámaras que constantemente estén constantemente identificando objetos, para evaluar desempeño del algoritmo de consenso.
- Poder ingresar cada cliente, los registros uno por uno, y visualizarlos en los monitores de los "workers n".
- El código no dependerá de algún contenedor, pero si puede usarlo para hacer sus pruebas.
- Usar hilos para mejorar el desempeño y evitar corrupción de registros.

Subir en Univirtual.

- Comprimido consta
 - o Códigos fuente de extensión en el LP1, LP2, LP3, ... (subir solo códigos fuente o explicativos como base, no subir otros archivos como historial, motor del lenguaje de programación u otros no necesarios)
 - o PDF Informe.
 - o PDF Presentación.
 - o Los grupos mayores a 2 alumnos desarrollaran un lenguaje de programación adicional LP3, ... para otro nodo.